ACTIVIDAD 3.2 Aplicación de servidor y clientes de chat

1. Objetivo

Demostrar conocimientos sobre desarrollo de aplicaciones compuestas por varios hilos, considerando los siguientes aspectos:

- Analizar el concepto de socket, sus tipos y características (RA3-d).
- Utilizar sockets para programar una aplicación cliente que se comunique con un servidor (RA3-e).
- Desarrollar una aplicación servidor en red y verificado su funcionamiento (RA3-f).
- Desarrollar aplicaciones que utilizan sockets para intercambiar información (RA3-g).
- Utilizar hilos para posibilitar la comunicación simultánea de varios clientes con el servidor (RA3-h).
- Caracterizar los modelos de comunicación más usuales en las arquitecturas de aplicaciones distribuidas (RA3-i).
- Depurar y documentar las aplicaciones desarrolladas (RA3-j).

2. Enunciado

En parejas, realizad la siguiente tarea competencial evaluable, relacionada con mecanismos de comunicación en red, que funcione como una herramienta de chat entre diferentes usuarios remotos (con características básicas):

1. Crear una aplicación Java con un modelo cliente/servidor que permita a los clientes mandar mensajes a otros clientes, como en un chat, enviándolos a través del servidor que actuará como rely.

2. El servidor debe:

   a) Admitir conexiones (sockets) para múltiples usuarios sin bloquearse, por un puerto que se pueda determinar mediante una constante.

   b) Notificar a un nuevo cliente qué otros clientes hay conectados al chat y notificar a todos los clientes ya conectados cuando un nuevo cliente se conecte.

   c) Ser capaz de recibir mensajes de un cliente, determinar a qué otro cliente van dirigidos y reenviarlos a dicho destinatario por la correspondiente conexión.

   d) Gestionar mensajes de un cliente dirigidos a todos los demás clientes, reenviando el mensaje a todos los demás usuarios exceptuando el que lo ha enviado.

   e) Los mensajes deben intercambiarse, entre clientes y servidor, mediante un objeto de tipo MensajeChat que contenga un identificador del origen, otro del destino, el nombre de usuario del origen (no se exigirá que los nombres de usuario sean únicos) y el texto del mensaje.

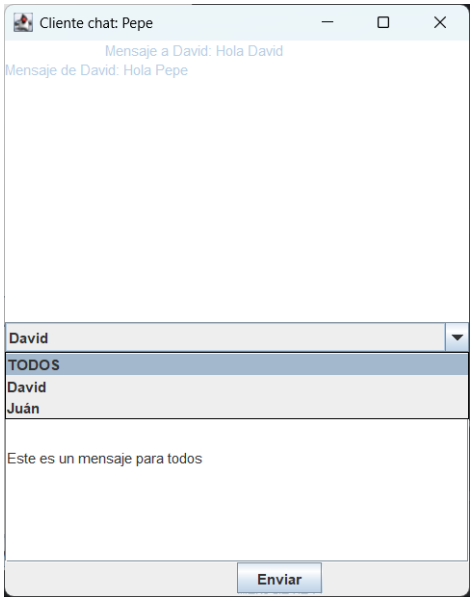
3. El cliente debe:

   a) Solicitar al usuario su nombre de usuario, la IP y el puerto por el que conectarse al servidor (se facilita la clase VentanaConfiguracion para su reutilización si se desea)

de manera similar a la imagen siguiente. La información será utilizada para establecer una conexión al servidor, mostrando un mensaje en el área de texto de la interfaz mencionada en el punto siguiente en caso de no ser posible realizar la conexión.

b) Cada cliente al ejecutarse debe mostrar una interfaz (similar a la de la imagen) con:

- Un área de texto en la que se muestren los mensajes recibidos de otros usuarios (identificando el usuario de origen) y los mensajes enviados por el usuario (identificando el usuario destino). Esta área no puede ser editada directamente por el usuario.
- Un desplegable que permita elegir a un usuario destinatario conectado o la opción TODOS.
- Un campo de texto en la que el usuario pueda introducir un mensaje de texto.
- Un botón enviar.



c) Cuando el usuario pulse el botón enviar, se deberá:

- Enviar el mensaje en el campo de texto (si no está vacío) al usuario seleccionado en el desplegable a través de la conexión con el servidor utilizando un objeto de tipo MensajeChat.
- En caso de haberse seleccionado la opción TODOS, se enviará un único mensaje configurado de modo que el servidor gestione el reenvío a todos los clientes conectados.
- Si el mensaje se envía con éxito, mostrará el mensaje en el área de texto.

d) La interfaz no debe bloquearse, permitiendo el uso de los componentes al mismo tiempo que atiende mensajes que lleguen a través de la conexión con el servidor.

e) Cuando se reciba un mensaje a través de la conexión con el servidor, en tiempo real, se mostrará el mensaje en el área de texto identificando el usuario origen del mensaje.

4. El código debe estar documentado mediante Javadoc, con la información de las clases, métodos y atributos, además de comentarios explicativos en el código.

5. Generad un fichero .zip con el contenido de la carpeta src del proyecto y subidlo al aula virtual con el nombre de fichero:

   DAM2_PSP_UT3_Actividad2_Nombres-alumnos

3. Evaluación

- La evaluación se realizará conforme a la siguiente lista de cotejo.
- La presentación de trabajos no genuinos o plagiados conllevará una calificación de 0 puntos en la actividad para todos los alumnos involucrados.
- La actividad deberá entregarse antes de la fecha de cierre configurada en el aula virtual para ser evaluada. De no ser así, se puntuará con un 0. Se reabrirá el plazo de entrega para las convocatorias de Ordinaria y Extraordinaria para aquellos alumnos que deban presentarse a ellas.
- Tras la entrega, el docente podrá realizar preguntas a los alumnos para evaluar su grado de contribución a la actividad.

Lista de Cotejo – Aplicación cliente/servidor de Chat

Criterio de evaluación	Sí	No	Observaciones
1. (1 punto) Las comunicaciones entre clientes y servidor utilizan sockets adecuadamente.	☐	☐	
2. (1 punto) Se pueden ejecutar varios clientes al mismo tiempo conectándose correctamente al servidor.	☐	☐	
3. (1 punto) Los mensajes entre clientes y servidor son intercambiados correctamente utilizando la clase MensajeChat definida conforme se requiere.	☐	☐	
4. (1 punto) Los mensajes se distribuyen correctamente al usuario identificado como destinatario, o a TODOS si así se selecciona.	☐	☐	

Criterio de evaluación	Sí	No	Observaciones
5. (1 punto) La interfaz gráfica del cliente contiene todos los componentes requeridos, muestra la información solicitada y permite interactuar correctamente al usuario.	☐	☐	
6. (1 punto) Al pulsar el botón enviar, el cliente genera y hace llegar el mensaje al servidor.	☐	☐	
7. (1 punto) Al recibir un mensaje del servidor el cliente lo procesa correctamente en tiempo real y lo muestra en el área de texto conforme se requiere.	☐	☐	
8. (1 punto) El servidor distribuye y los clientes actualizan convenientemente la información sobre los usuarios conectados.	☐	☐	
9. (1 punto) El programa es capaz de transmitir los mensajes entre los clientes a través del servidor sin que surjan excepciones o errores no controlados.	☐	☐	
10. (1 punto) El código está documentado correctamente con Javadoc y comentado.	☐	☐	

Puntuación máxima: 10 puntos.